

Density estimation

Chapter resources

[Slides \(PDF\)](#)[Open in Colab](#)[Practice quiz](#)

We have seen that clustering algorithms aim to mine the underlying structure of data by breaking it into coherent groups. For example k-means clustering **allows to identify parts of the feature space with high density (i.e., a large number of observations)** by partitioning all observations into a **discrete set of K data groups distributed around centroids**.

An alternative **smoother** approach would be to study the probability $P(X)$, where X is the data. Indeed, if we could determine the probability value of an given point in space $\mathbf{x}$, we could naturally discover clusters in the data (**zones with high probability values**). Knowing the distribution of the data is also useful for a number of things:

- Similar to clustering, if X is a series of observations of customers in a bank (e.g., age, sex, balance, salary, etc.), we may want to know $P(X)$ to understand which types of customers are more frequent (those that have large $P(X)$ values), or if there are different distinct groups of customers (e.g., if $P(X)$ has more than one mode);
- If X is a set of images of faces, knowing $P(X)$ would allow us to understand if an image x looks like an image of a face or not, which faces are more frequent, if there are more than one “groups” of faces, etc. If we could draw $x \sim P$, we could even generate new images of faces! (**by the way, this is what Generative Adversarial Networks do**);
- If X is a series of observations of bank transactions, by modeling $P(X)$ we can infer if a given observation $\mathbf{x}$ is a typical transaction (high $P(\mathbf{x})$) or whether it is an anomaly (low $P(X)$). If we can make assumptions on the nature of $P(X)$, we can even make inference on the properties of a given data point $\mathbf{x}$. For instance, if P is Gaussian, then datapoints which are further from the center are atypical and they can hence identify anomalous transactions.

A probability density is a **continuous function**, so estimating it is not as straightforward as estimating a **probability mass function**. There are two main classes of methods for density estimation:

- **Non-parametric methods**: these methods aim to estimate the density **directly from the data, without strong assumptions on the source distribution**. The main advantage of these methods is that they generally have few hyper-parameters to tune and can be used when no assumption can be made on the source of the data. A disadvantage is that, while we can numerically compute $f(\mathbf{x})$ with these methods (where f is the density function), we do not know the analytical form or property of f ;
- **Parametric methods**: these methods aim to **fit a known distribution to the data**. An example of this approach is **fitting a Gaussian to the data**. The main advantage of these methods is that they

provide an analytical form for the density f , which can be useful in a number of contexts (for instance, the density may be part of a cost function we want to optimize). The downside is that these methods make strong assumptions on the nature of the data.

In the following, we will see the main representatives of these two classes of methods.

Non-parametric Density Estimation Techniques

We will now look at **non-parametric methods**, which estimate the density *directly from the data* without making strong assumptions (like that the data is Gaussian).

We will focus on the two main approaches:

1. **Fixed Windows (Histograms):** This method “discretizes” the feature space into a fixed grid of bins and counts how many points fall into each bin.
2. **Mobile Windows (Kernel Density Estimation):** This method centers a “window” on *every single data point* (or on a grid) and sums their influence, creating a much smoother estimate. This is also known as the **Parzen Window** method.

Method 1: Fixed Windows (ND Histograms)

This is the simplest method, extending the 1D histogram to multiple dimensions. We “discretize” the feature space into a fixed grid of “bins” (or “tiles”) and count how many points fall into each bin.

A simple *count* is misleading. A bin that is 4x larger might have 4x more points but have the same *density*.

To get a true **probability density function** $f(\mathbf{x})$, our function must satisfy one rule: the **total “volume” under the function must integrate to 1**.

$$\int f(\mathbf{x}) d\mathbf{x} = 1$$

To achieve this, we cannot just plot the counts. We must define the density $f(R_i)$ for a bin R_i as the *probability* of that bin divided by its *volume* (or area).

1. **Probability of the bin:** $P(R_i) = \frac{\text{Count of points in } R_i}{\text{Total Number of Points } (N)}$
2. **Volume of the bin:** $V(R_i)$ (e.g., for a 2x2 bin, the area is 4).

This gives us the formula for the density *height* of that bin:

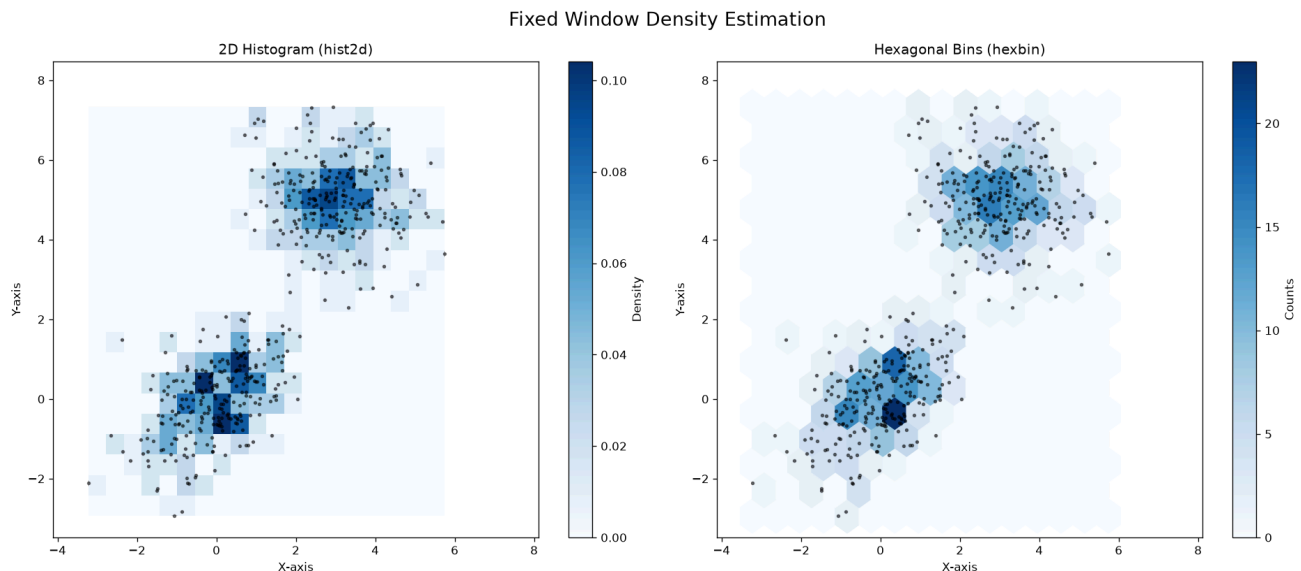
$$f(\mathbf{x} \in R_i) = \frac{P(R_i)}{V(R_i)} = \frac{|R_i|/N}{V(R_i)}$$

By dividing by the volume, we ensure that a small, crowded bin (small $V(R_i)$) will have a high density value, while a large, sparse bin (large $V(R_i)$) will have a low density value, even if they

contain the same number of points. This normalization is what ensures the total volume integrates to 1:

$$P(\mathbf{x} \in \cup_i R_i) = \sum_i P(R_i) = \sum_i f(\mathbf{x} \in R_i) V(R_i) = \sum_i \frac{|R_i|/N}{V(R_i)} V(R_i) = \frac{1}{N} \sum_i |R_i| = 1$$

This approach has two main hyperparameters: the **bin size** and the **grid's origin (position)**. A popular alternative to square bins is **hexbin**, which uses hexagonal bins that tile the space more naturally.



As you can see, both plots do a good job of finding the two high-density “blobs” in our data. The **hexbin** plot (right) often looks a bit smoother and more natural than the **hist2d** (left).

The main problem with “fixed window” methods is that the *bin size* is a difficult hyperparameter to choose. More importantly, the rigid, fixed grid is sensitive to where the points fall (the “bin edge” problem).

While we saw this in 2D, we can generalize it to N dimensions (e.g., squares become cubes or hypercubes).

Method 2: Mobile Windows (Kernel Density Estimation)

The “fixed window” (histogram) method has two main problems:

1. The “bin edges” are arbitrary and can change the shape of the density.
2. The resulting density is “blocky” and not smooth.

A more flexible and powerful non-parametric approach is to use a “**mobile window**”. This method is known as **Kernel Density Estimation (KDE)** or the **Parzen Window** method.

Instead of putting a fixed grid over the data, KDE places a “window” (or **kernel**) at each point in space $\mathbf{x}$ and asks, “How much data is inside this window?”

The “Naive” Approach: The Circular Kernel

A simple approach is to center a circle (or hypersphere) of a fixed radius h at a point $\mathbf{x}$ and calculate the density based on how many data points $N(\mathbf{x}, h)$ fall inside it.

This follows the same logic as the histogram: $\text{Density} = (\text{Probability of being in the window}) / (\text{Volume of the window})$

$$f(\mathbf{x}) = \frac{|N(\mathbf{x}, h)|/N}{V(h)}$$

where N is the total number of points, $|N(\mathbf{x}, h)|$ is the count of points inside the circle

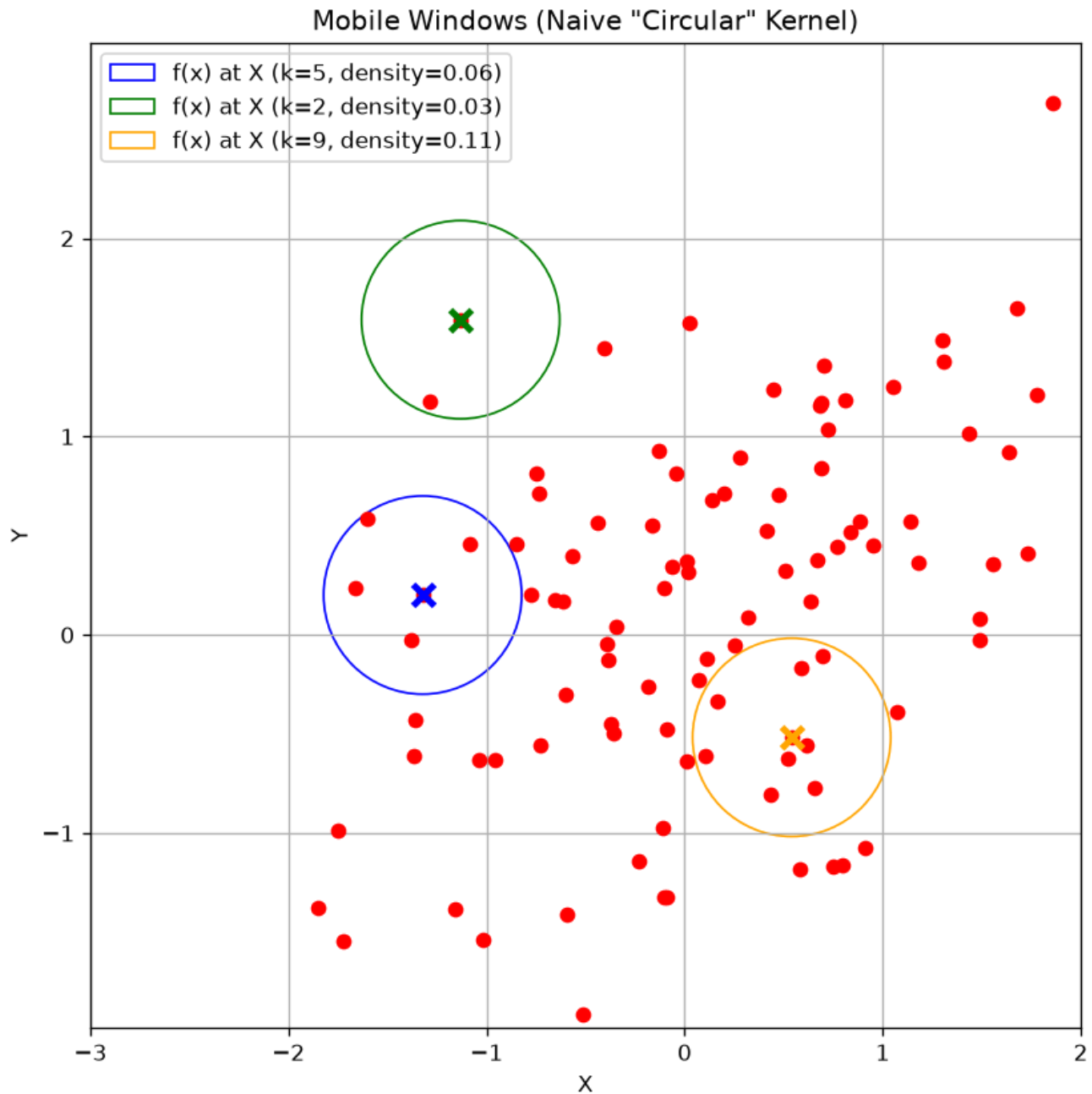
$$N(\mathbf{x}, h) = \{\mathbf{x}' \in \mathbf{X} \text{ s.t. } \|\mathbf{x}' - \mathbf{x}\|_2 \leq h\}$$

and $V(h)$ is the volume (or area) of the circle (e.g., $V(h) = \pi h^2$).

It is not hard to see that under the definition above, we have:

$$\int_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}) d\mathbf{x} = 1$$

The plot below shows some density estimates using this method:



Kernel View

We can also write the expression above as:

$$f(\mathbf{x}) = \frac{1}{|\mathbf{X}|} \sum_{i=1}^{|\mathbf{X}|} K_h(\mathbf{x}_i - \mathbf{x})$$

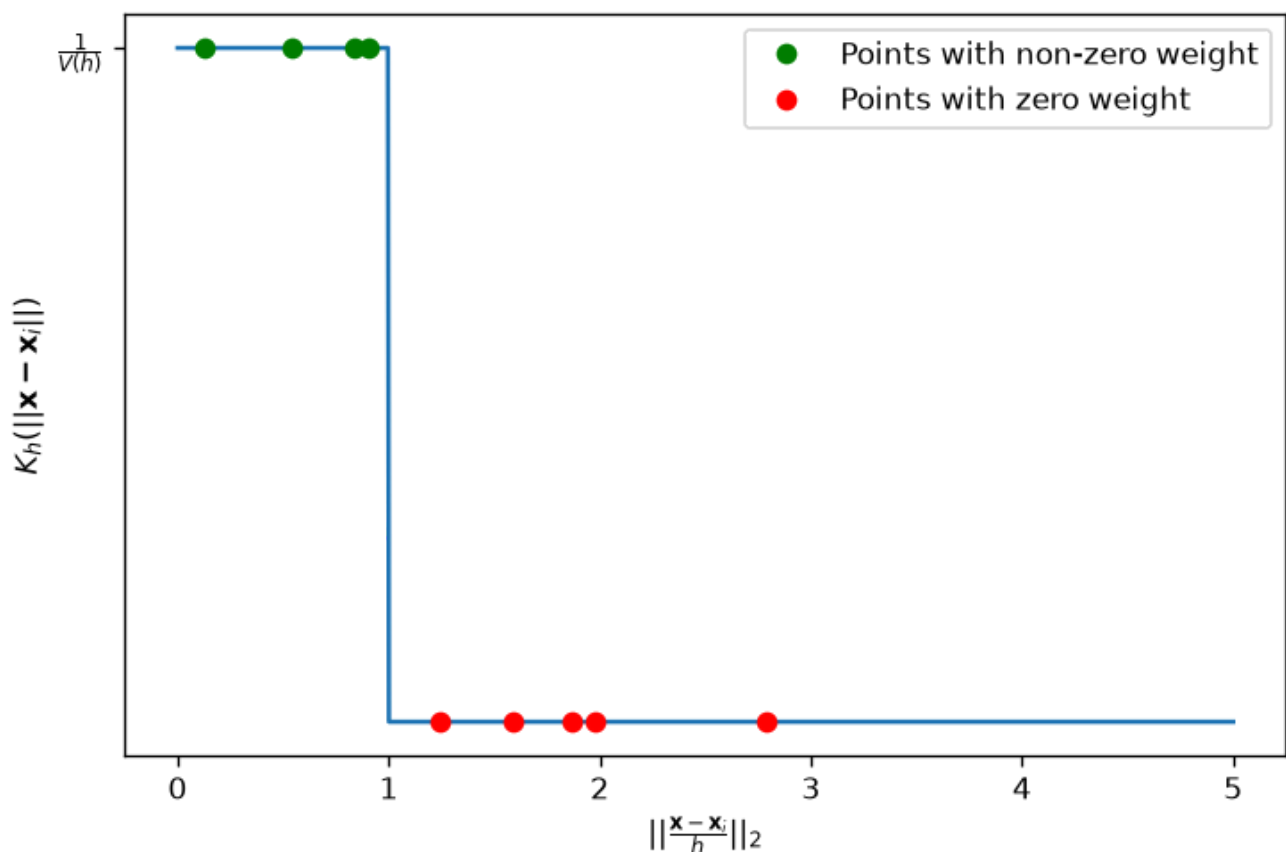
where $K_h \triangleq$ is defined as:

$$K_h(\mathbf{x}_i - \mathbf{x}) = \begin{cases} \frac{1}{V(h)} & \text{if } \|\frac{\mathbf{x}_i - \mathbf{x}}{h}\|_2 \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

We will call K_h a **kernel function** depending on the bandwidth parameter h . This is a function which assigns a weight to a point $\mathbf{x}_i$ depending on its distance from $\mathbf{x}$. In the example above, we chose K_h as a **circular (or radial) kernel** which assigns a uniform score to all points falling in a circle of radius h centered at $\mathbf{x}$.

A main problem with this density estimation approach is that **it can be very sensitive to the location at which we are computing the density**. Indeed, we can find cases in which we obtain very different densities when we move the circle by a little bit.

We can note that **this is due by the kernel K making “hard decisions” on which elements to assign a non-zero score and which ones to assign a zero score**. Indeed, if we plot the kernel as a function of the distance between $\mathbf{x}$ and $\mathbf{x}_i$, rescaled by h , we obtain the following picture:



The figure above plots points putting their distance from $\mathbf{x}$ in the x axis and the assigned weight in the y axis. As can be noted, even very close points in the x axis (e.g., the last of the green ones and the first of the red ones) get assigned very different weights, which makes the overall process sensitive to small shifts in $\mathbf{x}$.

The Problem with the “Naive” Kernel

This “circular” (or “radial”) kernel has a major flaw: it makes **“hard decisions”**. A point is either 100% “in” the circle (and gets a full vote) or 100% “out” (and gets a 0 vote).

This means that if you move the center $\mathbf{x}$ just a tiny bit, a data point might fall out of the circle, causing the density to “jump” abruptly. This creates a non-smooth, “bumpy” density estimate.

The Solution: “Smooth” Kernels

To fix this, we use a **smooth kernel** that assigns decreasing weight to points as their distance from the center $\mathbf{x}$ grows. The **Gaussian Kernel** is a reasonable choice:

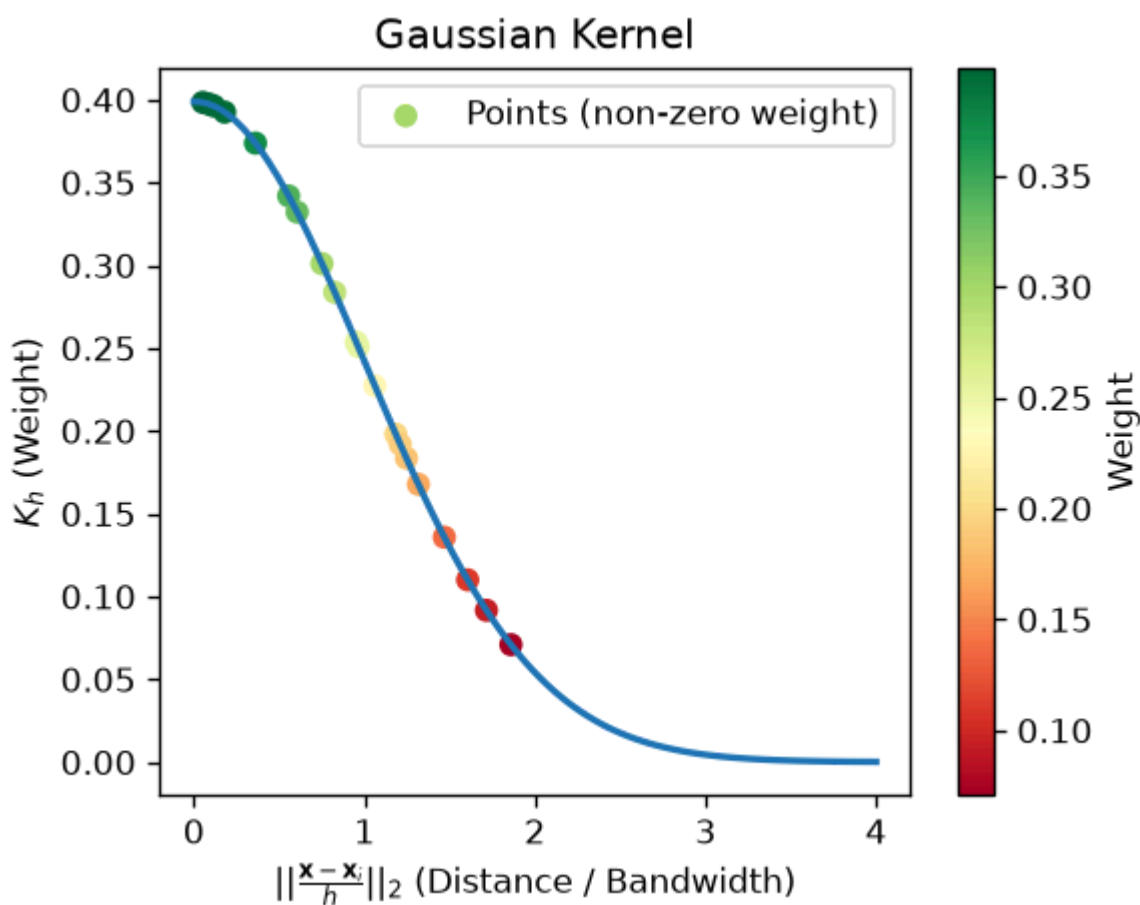
$$K_h(\mathbf{x}_i - \mathbf{x}) \propto \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}\|^2}{2h^2}\right)$$

With a smooth kernel, all points contribute to the density at $\mathbf{x}$, but nearby points contribute *much more* than faraway points.

The general formula for Kernel Density Estimation is a “sum of kernels,” one for each data point:

$$f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N K_h(\mathbf{x}_i - \mathbf{x})$$

We can visualize this kernel as follows:



The Epanechnikov Kernel

The Gaussian Kernel solves the sensitivity problem. However, it is very expensive to compute. Indeed, for every new density estimate, we have to compute and sum Gaussian terms over the whole dataset. This can be expensive if we have a large dataset.

An alternative is to set to zero all elements which are far enough. This saves a lot of computation.

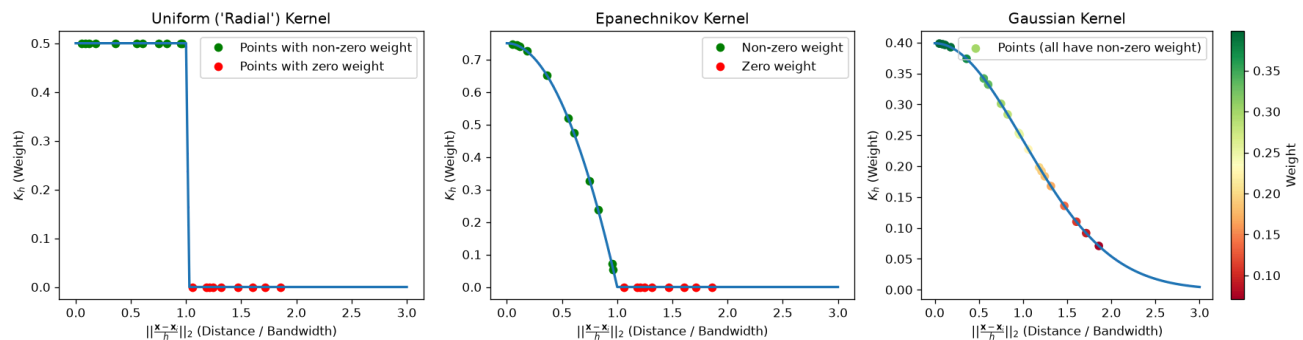
This is exactly what the Epanechnikov does. It is defined as follows:

$$K_h(\mathbf{x}_i - \mathbf{x}) = \frac{3}{4h^2} \left(1 - \frac{\|\mathbf{x}_i - \mathbf{x}\|^2}{h^2} \right) \mathbb{I}(\|\mathbf{x}_i - \mathbf{x}\| \leq h)$$

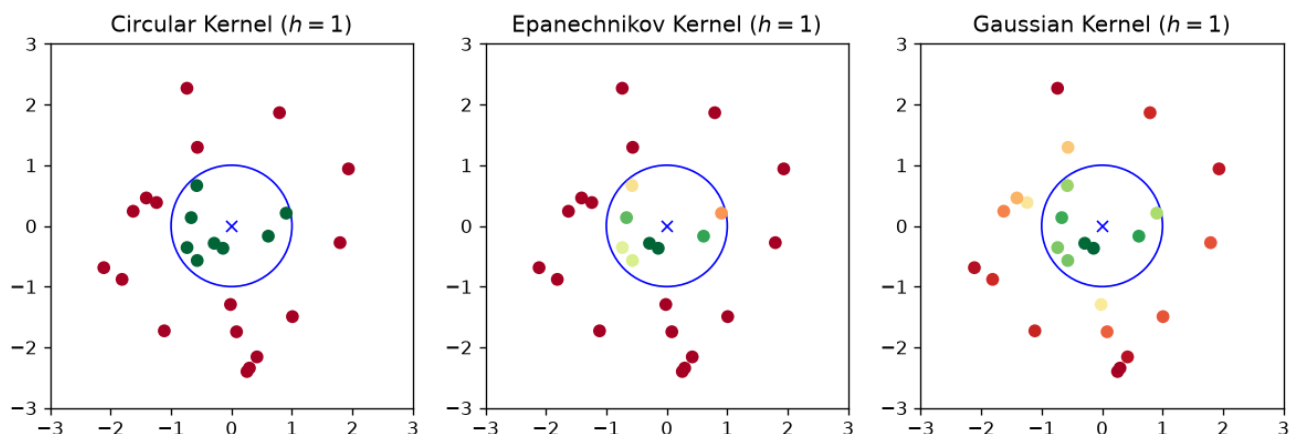
where $\mathbb{I}$ is the indicator function defined as:

$$\mathbb{I}(x) = \begin{cases} 1 & \text{if } x \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

The plot below compares the scores assigned to points depending on their distance from $\mathbf{x}$ according to the three kernels seen so far:



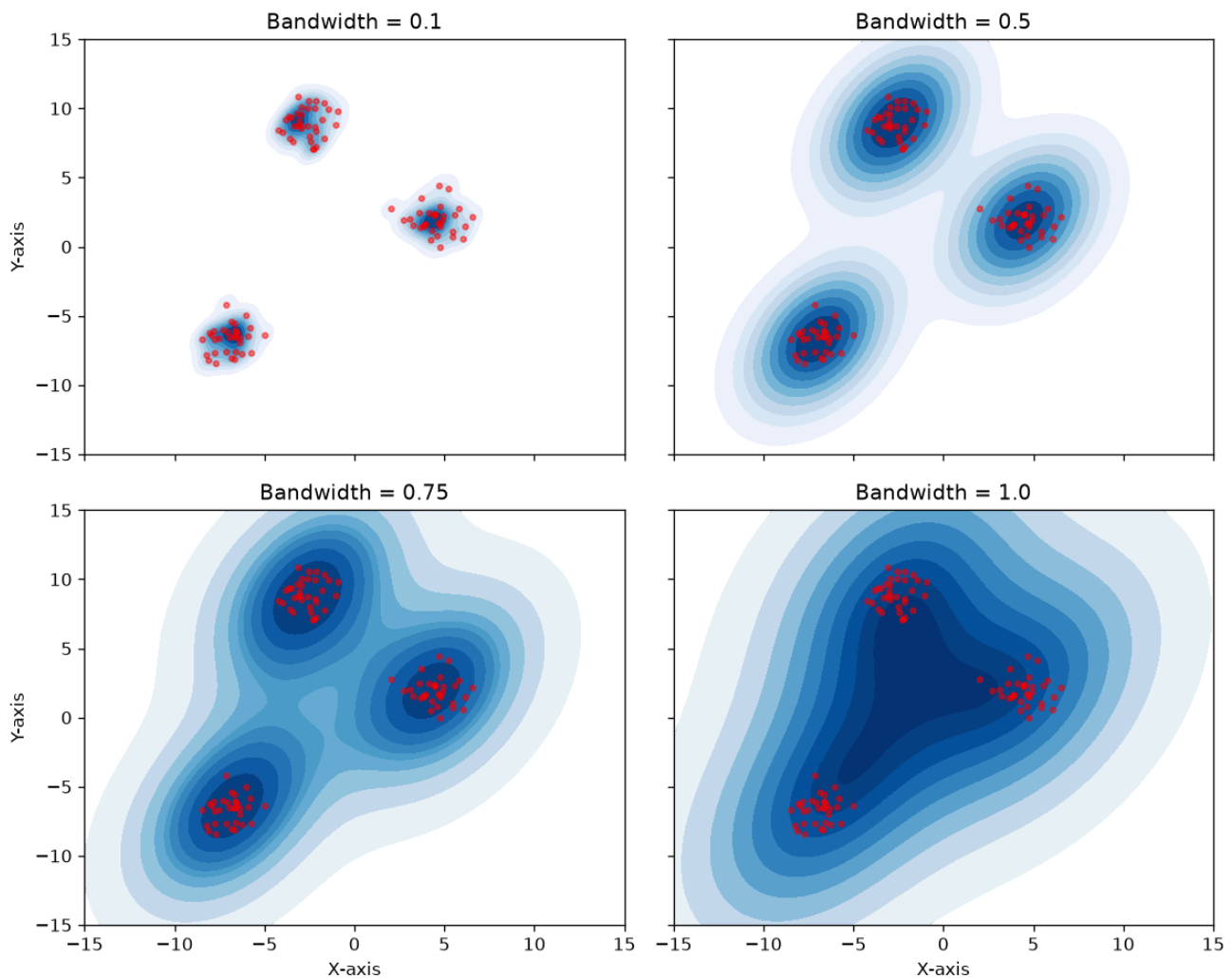
The effect of the kernels on the density estimations processes can be seen in the following plot:



The Bias-Variance Tradeoff: Choosing the Bandwidth h

The kernel shape (e.g., Gaussian) is not the most important choice. The **bandwidth (h)** is the *single most important hyperparameter* in density estimation. It controls the **bias-variance tradeoff**.

- Small h (e.g., `bandwidth = 0.1`):
 - Low Bias, High Variance.
 - The model “trusts” only a tiny local area. The resulting density is “spiky” and “noisy.” It **overfits** to the training data.
- Large h (e.g., `bandwidth = 1.0`):
 - High Bias, Low Variance.
 - The model “blurs” the data by averaging over a huge area. The resulting density is “over-smoothed.” It **underfits**, hiding the true structure (like the two separate clusters).



Final Result: KDE

As we can see, choosing a good bandwidth (like $h = 0.5$ or $h = 0.75$) is critical.

When we use a good kernel (like Gaussian) and a well-chosen bandwidth, we can produce a smooth, accurate, and non-parametric estimate of the true data density. The `seaborn.kdeplot` function does all of this for us automatically.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Generate random 2D points
np.random.seed(42)
data = np.random.multivariate_normal([0, 0], [[1, 0.5], [0.5, 1]], size=250)
data2 = np.random.multivariate_normal([3, 5], [[1, -0.2], [-0.2, 1]], size=250)

data = np.concatenate([data, data2])

# Set the size of the bins (adjust as needed)
bin_size = 0.5

# Calculate the range of the plot
x_min, x_max = data[:, 0].min() - 1, data[:, 0].max() + 1
y_min, y_max = data[:, 1].min() - 1, data[:, 1].max() + 1

# Calculate the number of bins in each dimension
num_x_bins = int((x_max - x_min) / bin_size)
num_y_bins = int((y_max - y_min) / bin_size)

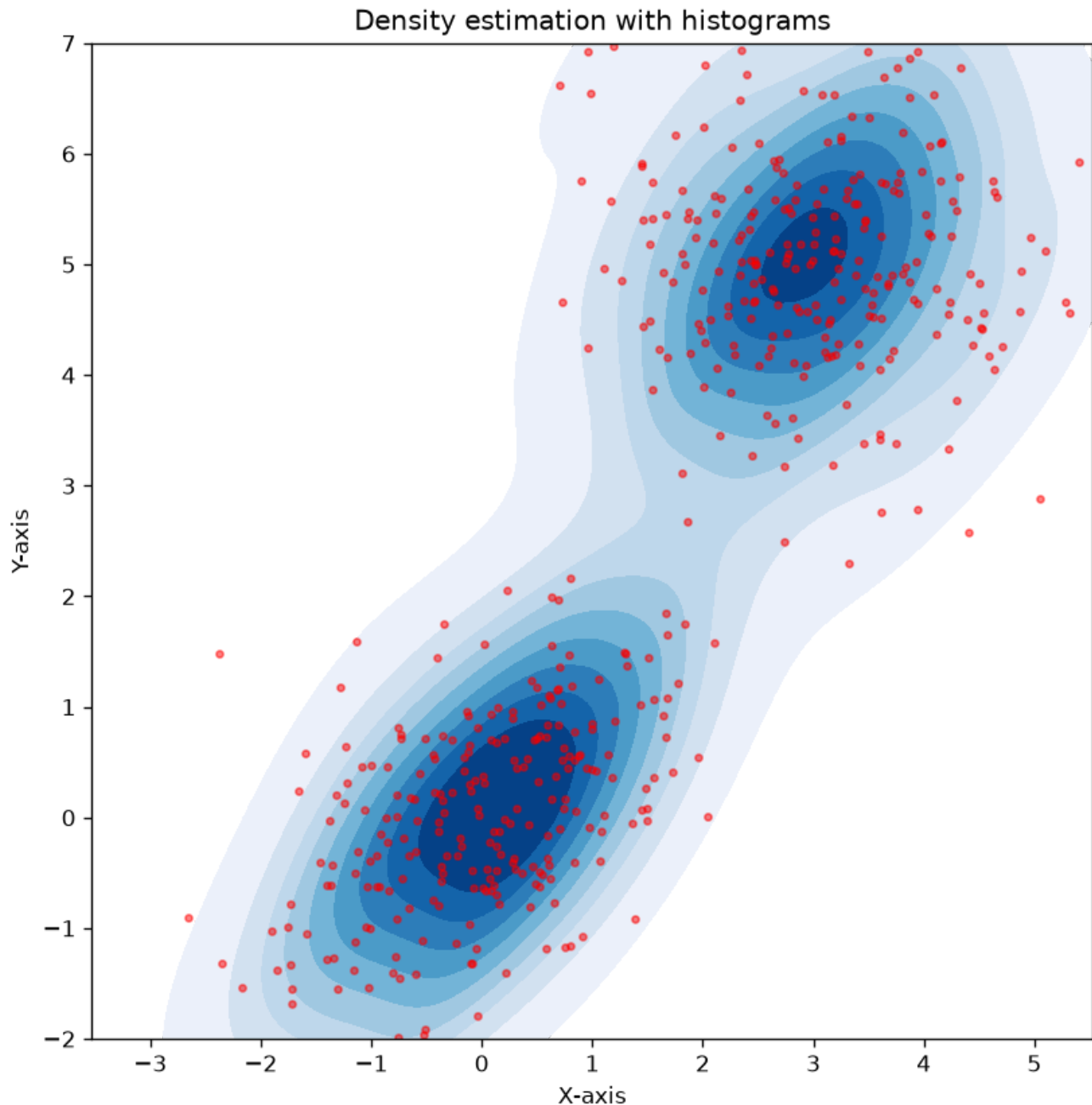
# Create a square bin plot
plt.figure(figsize=(8, 8))
plt.plot(data[:,0],data[:,1],'.r', alpha=0.5)
sns.kdeplot(x=data[:, 0], y=data[:, 1], cmap='Blues', fill=True)

# Set labels and title
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.axis('equal')
plt.title('Density estimation with histograms')

plt.xticks(np.arange(-3,8))
plt.yticks(np.arange(-3,8))
plt.xlim([-3,5])
plt.ylim([-2,7])
#plt.grid()

# Display the plot
plt.show()
```





Parametric Density Estimation

Parametric methods aim to estimate density by **fitting a parametric model to the data**. This has some advantages over non-parametric density estimation:

- **The chosen model has in general a known analytical formulation which we can reason on**, or that we can even plug into a cost function and differentiate to pursue some optimization objective;
- **The chosen model is compact**. While non-parametric models need to keep in memory the whole dataset to make density estimations at given points, once fitted, parametric models can be used to predict density values using the analytical formulation of the model;
- **If we choose an interpretable model, we can reason on the model to derive properties of the data**. For instance, if we fit a Gaussian on the data, then we know that the mean is the point with

highest density. We also know that when we go further from the mean, the density values decrease;

- **Parametric approaches impose constraints and hence work better when we do not have much data.** Non-parametric approaches, instead, make very little assumptions on the data, so they are better suited to the cases in which we have large datasets which are good representatives of the population.

Nevertheless, these methods also have disadvantages:

- **An optimization process is required** to fit the model to the data, i.e., find appropriate values which make the model “explain well the data”. This can be a time-consuming process.
- Parametric models make **strong assumptions on the data**. For instance, if we fit a Gaussian to the data, we assume that the data is distributed in a Gaussian way. If this is not true, then the model will not be very accurate. This is a relevant point, as in many cases we cannot make strong assumptions on the data.

For the reasons above, **it is common to use non-parametric density estimations for visualizations** (e.g., the density plots we have seen and used in many cases), especially for exploration, when we do not know much about the data, and to use parametric model when we need to create efficient models able to make predictions (as we will see later in the course).

Fitting a Gaussian to the Data

The most common parametric method is to assume our data follows a **Multivariate Gaussian (Normal) Distribution**.

Recall the PDF (Probability Density Function) for a d -dimensional Gaussian:

$$N(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^d \det(\boldsymbol{\Sigma})}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

This entire, complex distribution is defined by only two parameters:

- The mean vector $\boldsymbol{\mu}$ (the center of the cloud).
- The covariance matrix $\boldsymbol{\Sigma}$ (the shape and orientation of the cloud).

Our task is to find the $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ that **best fit our data $\mathbf{X}$** .

Maximum Likelihood Estimation (MLE)

Let $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \triangleq$ be a set of observations which we will assume to be **independent and drawn from a multivariate Gaussian distribution of unknown parameters**. We would like to find the parameters $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ that identify the **most likely Gaussian distribution that may have generated the data**. To do so, we can use the **Maximum Likelihood (ML)** principle, which consists in finding the parameters which maximize the probability values that the Gaussian distribution would compute for the given data. The likelihood is defined as

$$P(\mathbf{X}|\mu, \Sigma) = \prod_i \mathcal{N}(\mathbf{x}_i|\mu, \Sigma)$$

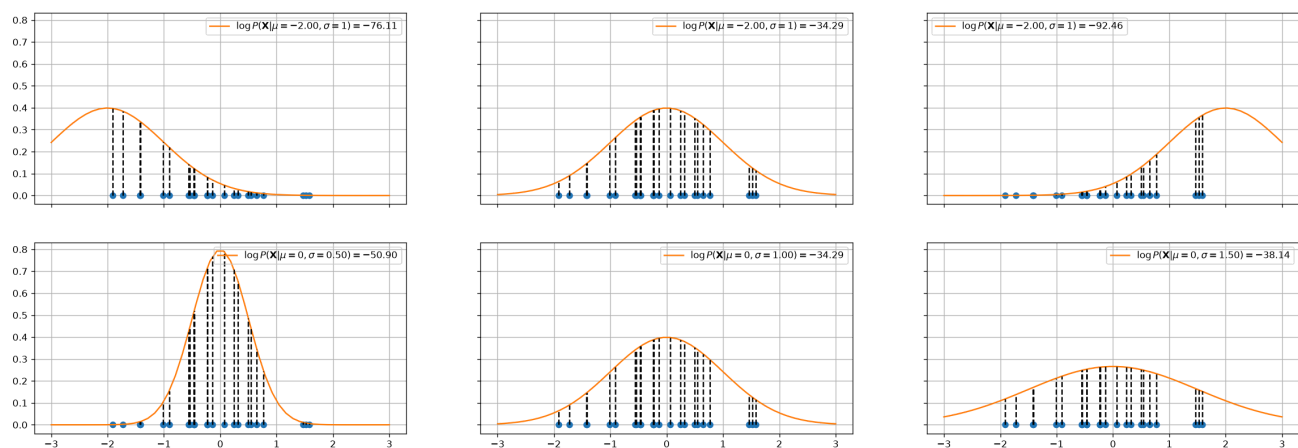
In practice, we maximize the log-likelihood instead.

Intuitively, this works as follows:

1. We “try” a specific Gaussian (a given μ and Σ).
2. We calculate the probability density (the “likelihood”) of *all* of our data points under this Gaussian.
3. We multiply these probabilities together to get the **Total Likelihood** of our entire dataset.
4. The goal is to find the *one* μ and Σ that **maximize this total likelihood**.

This sounds like a complex optimization problem, but for a Gaussian distribution, a closed-form solution exists. It can be shown by taking the derivative of the log-likelihood (the log of the likelihood function) and setting it to zero.

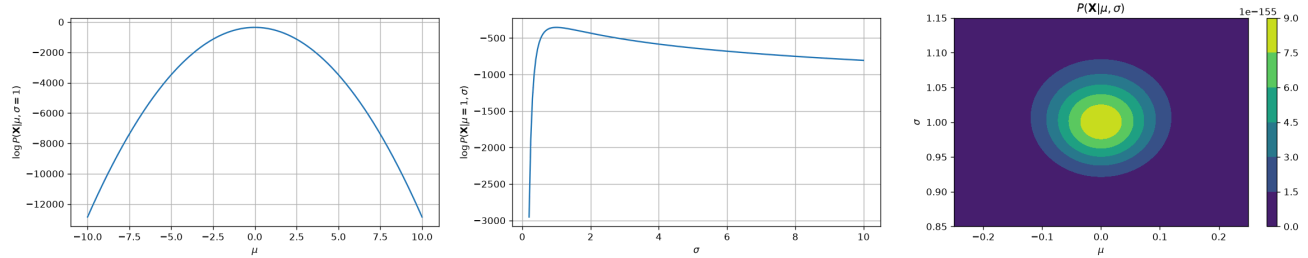
Let us consider a simple example in which we want to fit a Gaussian to a set of 1D data. The plot below shows the log likelihood for Gaussians with different means and standard deviations.



In the top plots, the standard deviation is fixed to 1, while the mean of the Gaussian is changed. The log likelihood is shown in the legend. As can be seen, the mean in the center plot leads to a larger log likelihood. **Intuitively, this happens when we choose a mean such that the majority of points fall in the central part of the Gaussian distribution.**

The plots in the bottom show a similar example in which the mean is fixed to 0 and the standard deviation is varied. As can be noted, when the standard deviation is too low (left) or too high (right), we obtain smaller log likelihoods.

The plots below show the log likelihood (first two plots) or the likelihood (last plot) as a function of different values of means and standard deviations.



The first two plots show how the log likelihood changes when μ and σ change, keeping the data fixed (same data as the previous plot). The last plot shows how the likelihood changes as a function of both μ and σ . As we can see, we have a maximum for $\mu = 0$ and $\sigma = 1$, which are the true parameters of the distribution which generated the data.

The Solution: Sample Mean and Covariance

The final result is incredibly intuitive. The Maximum Likelihood estimates for the Gaussian parameters are simply:

1. The ML estimate for the mean (μ_{ML}) is the sample mean:

$$\mu_{ML} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

2. The ML estimate for the covariance (Σ_{ML}) is the sample covariance matrix:

$$\Sigma_{ML} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \mu_{ML})(\mathbf{x}_i - \mu_{ML})^T$$

(Note: In statistics, you often see a $1/(N - 1)$ denominator for an “unbiased” estimate. The $1/N$ is the true Maximum Likelihood estimate, but in practice, they are nearly identical.)

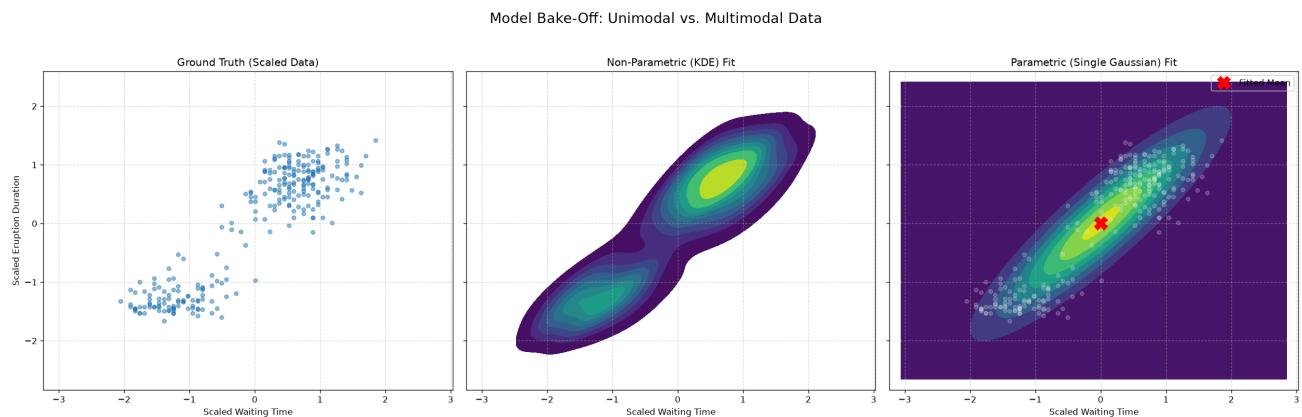
In short: “Fitting a Gaussian” is just a fancy way of saying “calculating the mean and covariance of your data.”

The Weakness of a Single Gaussian: Bimodal Data

The single Gaussian model we just learned is powerful, but it’s based on one very strong assumption: that our data comes from a *single cluster* (i.e., it is **unimodal**).

This assumption fails for many real-world datasets. A perfect example is the “Old Faithful” **geyser** dataset, which we know from our clustering lecture is **bimodal** (it has two distinct clusters).

What happens if we try to fit our simple parametric model (a single Gaussian) to this bimodal data? Let’s compare it to the non-parametric KDE, which we know can find the true shape.



The result is clear:

- The **KDE plot (middle)** correctly finds the two high-density clusters. It is flexible and *non-parametric*, so it doesn't care that the data isn't a single "blob."
- The **Single Gaussian plot (right)** *fails*. It places its mean (the red 'X') in the empty, low-density space *between* the two clusters. It's a "unimodal" model trying to fit "bimodal" data.

This proves the limitation of our simple parametric model. To solve this, we need a more powerful parametric model that can *learn* to model multi-modal data.

Gaussian Mixture Models (GMM)

Our comparison showed a critical weakness: our simple parametric model, a single Gaussian, failed to capture the **bimodal** (two-cluster) structure of the `geyser` data. The non-parametric KDE *did* find the shape, but it doesn't give us a compact, mathematical model.

This leads to the obvious question: Can we create a *parametric* model that is flexible enough to handle multi-modal data?

Yes. The solution is a **Gaussian Mixture Model (GMM)**.

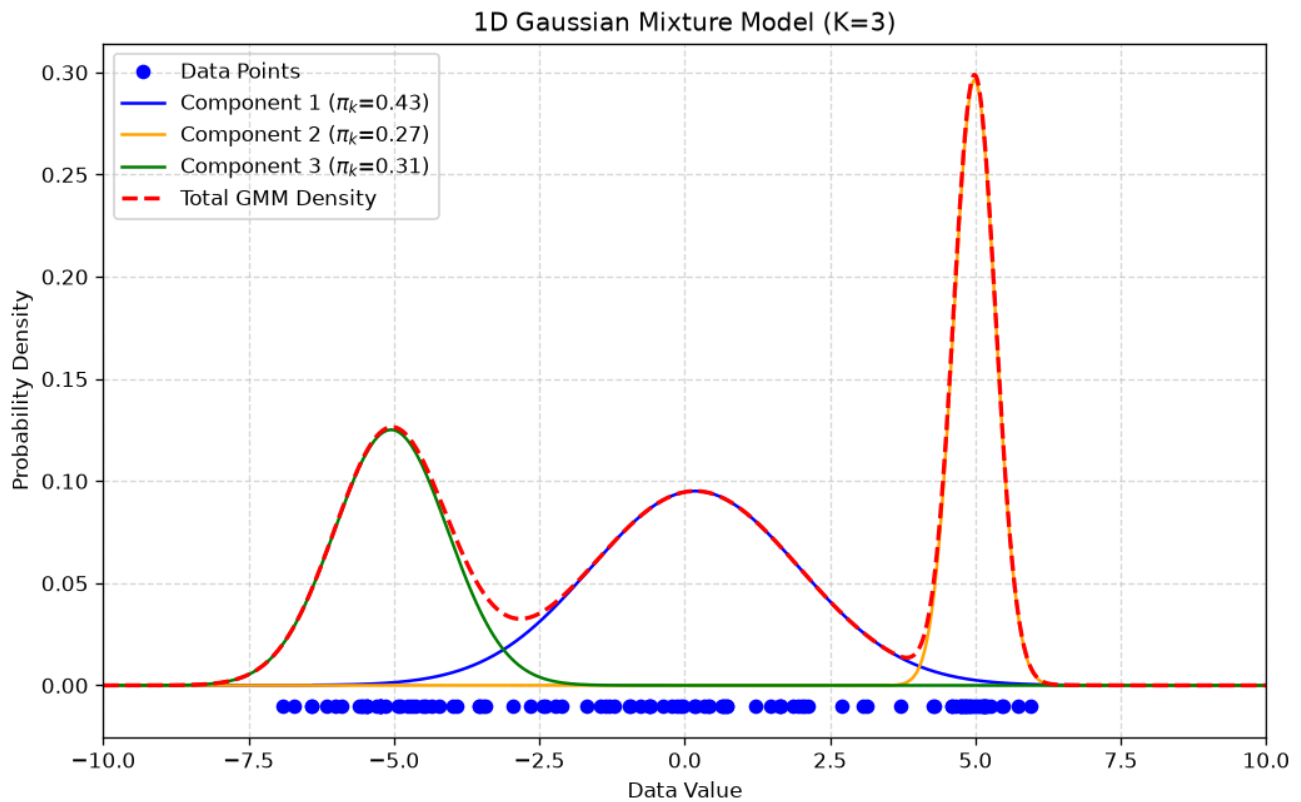
The idea is simple: instead of one Gaussian, we model the data as a **weighted average (a "mixture") of K different Gaussian components**.

$$P(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

This model is defined by three sets of parameters:

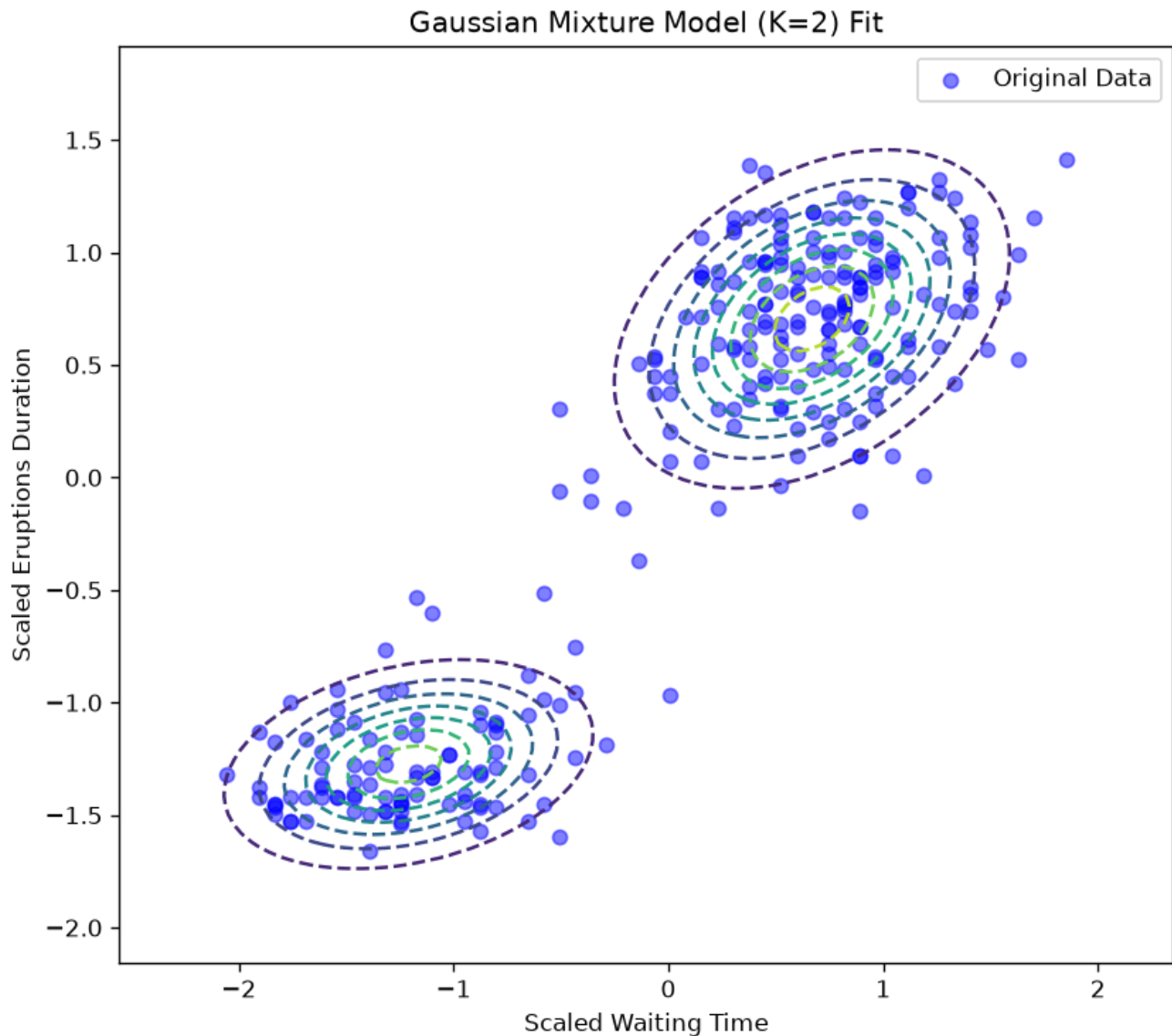
- π_k (Mixing Coefficients): The "weight" or prior probability of each component. (e.g., $\pi_1 = 0.6, \pi_2 = 0.4$). Must sum to 1.
- μ_k (Means): The center of each of the K Gaussian components.
- Σ_k (Covariances): The shape and orientation of each of the K components.

The 1D plot below shows a GMM with $K = 3$. The final density (red-dashed) is a *mixture* of three separate Gaussian components (blue, orange, green), allowing it to model complex, multi-modal data.



GMM Density Estimation Example

Now let's apply this to our `geyser` data. When we fit a GMM with $K = 2$, it should succeed where the single Gaussian failed.



Success! The GMM has learned two distinct Gaussian components that perfectly model the two clusters in our data.

GMM as “Soft Clustering”

GMM can be seen as a “soft” version of K-Means. If K-Means makes a hard assignment of data points to clusters, it can be shown that GMM does it in a “soft” way, computing instead the probability of assigning an element to a cluster.

It does this by introducing a **latent variable**, Z encoding the event:

The example $\mathbf{x}$ belongs to a given cluster.

In practice, $Z = k$ denotes the case in which $X = \mathbf{x}$ belongs to cluster k .

A **latent variable** is a “hidden” variable for which we don’t observe ground-truth values, but which we

assume is intrinsically part of the data. In this GMM, for every data point $\mathbf{x}_i$, there is a hidden z_i

that “indicates” which of the K Gaussian components *generated* that point.

To compute the soft assignment, we find the **posterior probability**, $P(Z = k|\mathbf{x})$: “What is the probability that this point $\mathbf{x}$ belongs to cluster k ?”

We solve this using **Bayes’ Theorem** to “flip” the question:

$$P(Z = k|\mathbf{x}) = \frac{P(\mathbf{x}|Z = k)P(Z = k)}{P(\mathbf{x})}$$

Once the GMM is trained (which we’ll see how to do later with the E-M algorithm), we *have* all the pieces for this formula:

1. $P(\mathbf{x}|Z = k)$ **(The Likelihood):**

- *In words:* “What is the probability of seeing $\mathbf{x}$ if it came from cluster k ?”
- *The value:* This is simply the value of our k -th Gaussian component: $\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$ $\triangleq$.

2. $P(Z = k)$ **(The Prior):**

- *In words:* “How common is cluster k in general?”
- *The value:* This is our mixing coefficient, π_k $\triangleq$.

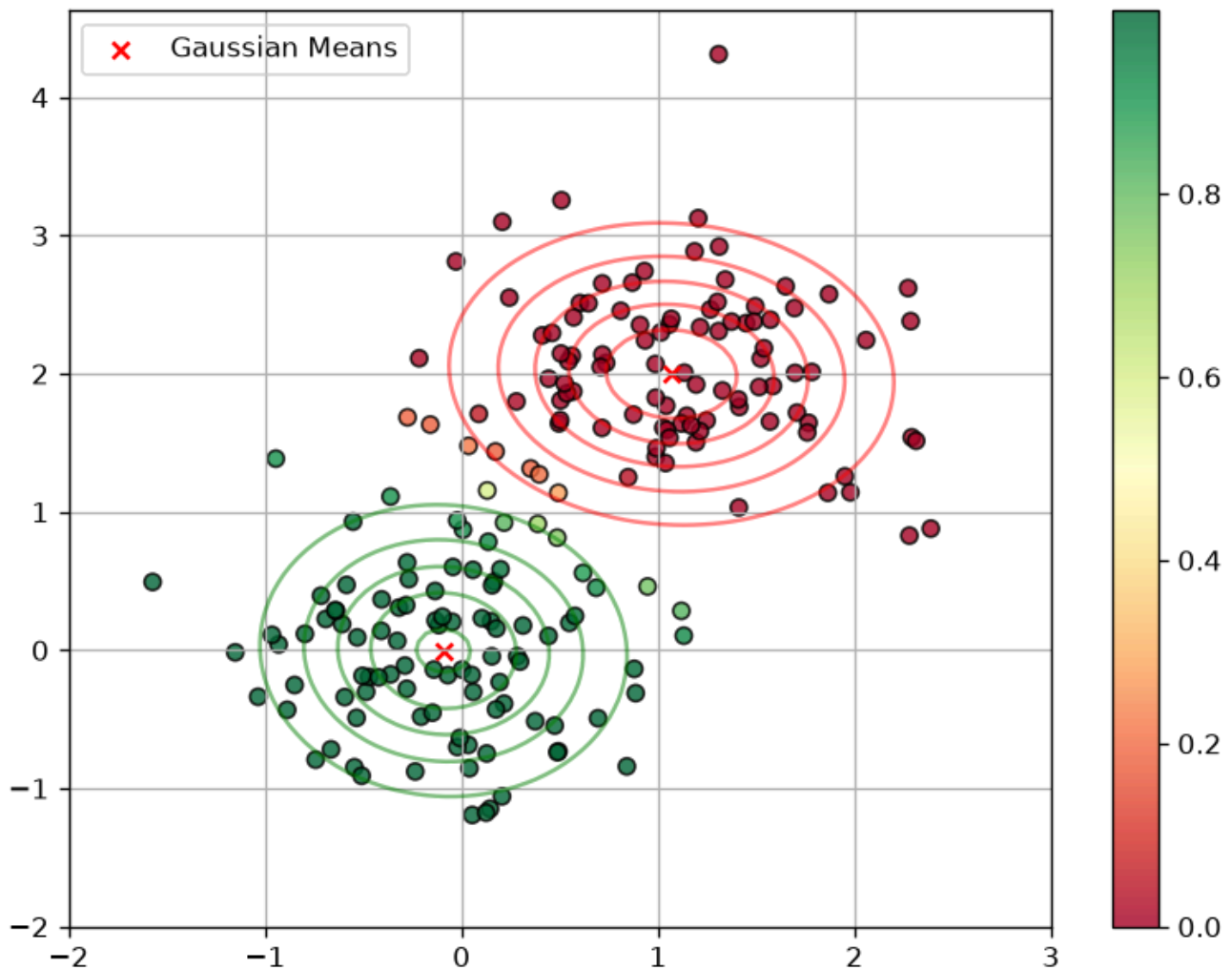
3. $P(\mathbf{x})$ **(The Evidence):**

- *In words:* “What is the total probability of seeing $\mathbf{x}$ under any cluster?”
- *The value:* This is just the sum of all components, which is our GMM formula: $\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)$ $\triangleq$.

Putting it all together, the “soft assignment” (or **responsibility**, γ_k $\triangleq$) for a point $\mathbf{x}$ belonging to cluster k is:

$$\gamma_k = P(Z = k|\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)}$$

This formula is exactly what is being used to generate the colors in the plot below. Points in the middle get an “uncertain” color (like yellow, ≈ 0.5) because their probability is split between the two components.



How to “Learn” a GMM: Optimization by Maximum Likelihood

We have defined our GMM as a sum of weighted Gaussians:

$$P(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)$$

The model's parameters are all the means, covariances, and mixing coefficients:

$$\theta = (\pi, \mu, \Sigma) = \{\pi_1, \dots, \pi_K, \mu_1, \dots, \mu_K, \Sigma_1, \dots, \Sigma_K\}$$

To “fit” the model, we use **Maximum Likelihood Estimation (MLE)**, just as we did for a single Gaussian.

We want to find the parameters θ that maximize the log-likelihood of our observed data

$$\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \triangleq:$$

$$\log P(\mathbf{X} | \theta) = \sum_{i=1}^N \log \left[\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k) \right]$$

This problem can not be optimized directly. Instead, a dedicated algorithm called Expectation-Maximization is used. We will not see this in details.

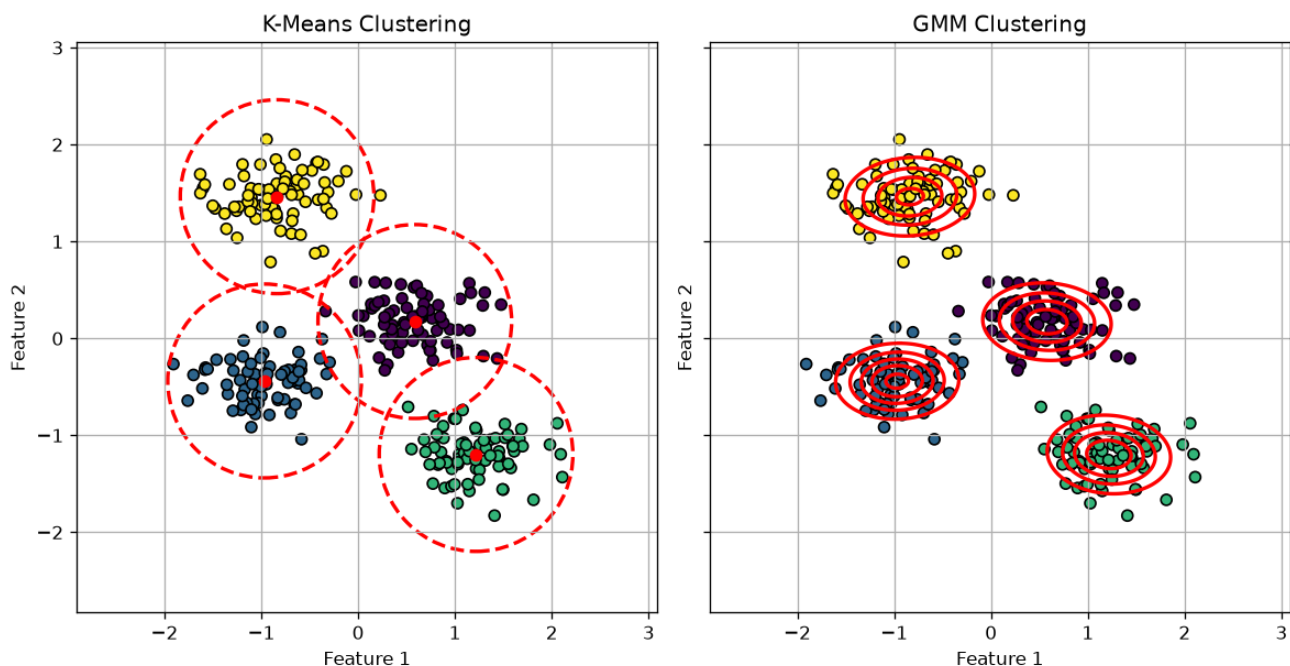
GMM vs K-Means

The EM algorithm is very similar to the optimization algorithm of K-Means. It can be shown that K-Means can be seen as a particular case of Gaussian Mixture Models in which:

- Points are assigned to clusters in a “**hard**” way, as compared to the **soft assignment** made by the responsibilities $\gamma(z_{ik}) \triangleq$ in Gaussian Mixture Models;
- Clusters are assumed to have diagonal covariance matrices $\epsilon \mathbf{I}$ where $\mathbf{I}$ is the $D \times D$ identity matrix and ϵ is a variance parameter which is **shared by all components**. This means that the clusters are assumed to be all **symmetrical and with the same shape**.

Despite these differences highlight the limitations of K-Means, it should be considered that K-Means is a significantly faster algorithm than GMM. Hence, if studying the probability density $P(\mathbf{X})$ is not needed and we can assume symmetrical clusters with similar shapes, K-Means is in practice a more popular choice, due to its faster convergence.

The plot below illustrates such differences:



Lab: Density Estimation for Anomaly Detection

In this lab, we will use our density estimation techniques to build a more complex **Anomaly Detector**.

The Goal

Our task is to build a model that can identify the digit ‘0’ as an “anomaly.”

1. **“Normal” Training Data:** We will train our models on a large dataset of “normal” digits (all images of ‘1’s, ‘2’s, ‘3’s, ... ‘9’s). This “normal” data is highly **multi-modal** (it has 9 distinct clusters).

2. **Test Data:** We will then create a test set containing a mix of “normal” digits (1-9) and “anomalous” digits ('0's).
3. **The “Bake-Off”:** We will compare three models to see which is best at this task:
 - **Model 1 (Single Gaussian):** A “naive” parametric model. We expect this to fail, as it will try to fit one “average” blob over all 9 normal digits.
 - **Model 2 (GMM):** A “smart” parametric model. We will tell it to fit $K = 4$ components, to capture some diversity in the data.
 - **Model 3 (KDE):** A “flexible” non-parametric model.

A good model will give a **high density score** to the “normal” digits and a **very low score** to the “anomalous” '0's.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import KernelDensity
from sklearn.metrics import roc_curve, auc
```



Step 1: Load and Prepare the Data

We'll load the `digits` dataset and split it into three groups:

1. `X_train_normal`: A training set of 70% of all “normal” digits (1-9).
2. `X_test_normal`: A test set of 30% of all “normal” digits (1-9).
3. `X_test_anomaly`: All of the “anomalous” digit '0's.

```
# 1. Load Data
X_all, y_all = load_digits(return_X_y=True)

# 2. Separate "normal" (1-9) from "anomaly" (0)
X_normal_all = X_all[y_all != 0]
y_normal_all = y_all[y_all != 0]

X_anomaly_all = X_all[y_all == 0]
y_anomaly_all = np.ones(len(X_anomaly_all)) # Label 1 = Anomaly

# 3. Create our Train/Test split
# We split the "normal" data into a train and test set
X_train_normal, X_test_normal, _, _ = train_test_split(
    X_normal_all, y_normal_all, test_size=0.3, random_state=42, stratify=y_normal_all
)
y_test_normal = np.zeros(len(X_test_normal)) # Label 0 = Normal

# 4. Create the final test set
X_test = np.concatenate((X_test_normal, X_anomaly_all))
y_test = np.concatenate((y_test_normal, y_anomaly_all))

print(f"Training data (Normal, 1-9): {X_train_normal.shape}")
print(f"Test data (Mixed): {X_test.shape}")
print(f"    → Normal '1-9's in test: {len(X_test_normal)}")
print(f"    → Anomalous '0's in test: {len(X_anomaly_all)}")
```

```
Training data (Normal, 1-9): (1133, 64)
Test data (Mixed): (664, 64)
    → Normal '1-9's in test: 486
    → Anomalous '0's in test: 178
```

Step 2: Preprocessing (Scaling)

We will fit a `StandardScaler` only on our `X_train_normal` data and use it to transform both our train and test sets.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_normal)
X_test_scaled = scaler.transform(X_test)
```

Step 3: Fit the Density Models

Now we will fit our three competitors to the `X_train_scaled` data (which contains 9 different clusters of digits).

```
# 1. Fit the Single Gaussian (Unimodal Parametric)
# This model will try to find the "average" of all 9 digits.
print("Fitting Single Gaussian (K=1)...")
model_single_gauss = GaussianMixture(n_components=1, random_state=42)
model_single_gauss.fit(X_train_scaled)

# 2. Fit the GMM (Multi-modal Parametric)
print("Fitting GMM (K=4)...")
model_gmm = GaussianMixture(n_components=4, random_state=42)
model_gmm.fit(X_train_scaled)

# 3. Fit the KDE (Non-Parametric)
# We'll use a bandwidth found from experimentation
print("Fitting KDE...")
model_kde = KernelDensity(kernel='gaussian')
model_kde.fit(X_train_scaled)

print("All three models are trained on the '1-9' digits.")
```

```
Fitting Single Gaussian (K=1)...
Fitting GMM (K=4)...
Fitting KDE...
All three models are trained on the '1-9' digits.
```

Step 4: Get Anomaly Scores

Now we'll use all three models to score our `X_test_scaled` data. We will use the **Negative Log-Likelihood (NLL)** as our “anomaly score.”

- `model.score_samples()` returns the log-likelihood (high = normal).
- `-model.score_samples()` returns the NLL.

With NLL, a **low score** means “this point looks normal,” and a **high score** means “this point looks weird (anomalous).” This is a more intuitive way to think of an “anomaly score.”

```
# Get the NLL scores for every point in the test set
# (NLL = -LogLikelihood)
scores_single = -model_single_gauss.score_samples(X_test_scaled)
scores_gmm = -model_gmm.score_samples(X_test_scaled)
scores_kde = -model_kde.score_samples(X_test_scaled)

# Store results in a DataFrame for easy plotting
results_df = pd.DataFrame({
    'y_true': y_test,
    'SingleGaussian_NLL': scores_single,
    'GMM_K4_NLL': scores_gmm,
    'KDE_NLL': scores_kde
})

results_df['Label'] = results_df['y_true'].map({0: 'Normal (1-9)', 1: 'Anomaly (0)'})
```

Step 5: Quantitative Comparison (The ROC Curve)

The histograms show GMM and KDE are much better, but the **ROC Curve** gives us the definitive quantitative score.

We will plot the **True Positive Rate** (finding the '0's) vs. the **False Positive Rate** (falsely flagging the '1-9's) for all three models. The **Area Under the Curve (AUC)** will be our final metric.

```
from sklearn.metrics import roc_auc_score

# --- Create the ROC Plot ---
plt.figure(figsize=(10, 8))

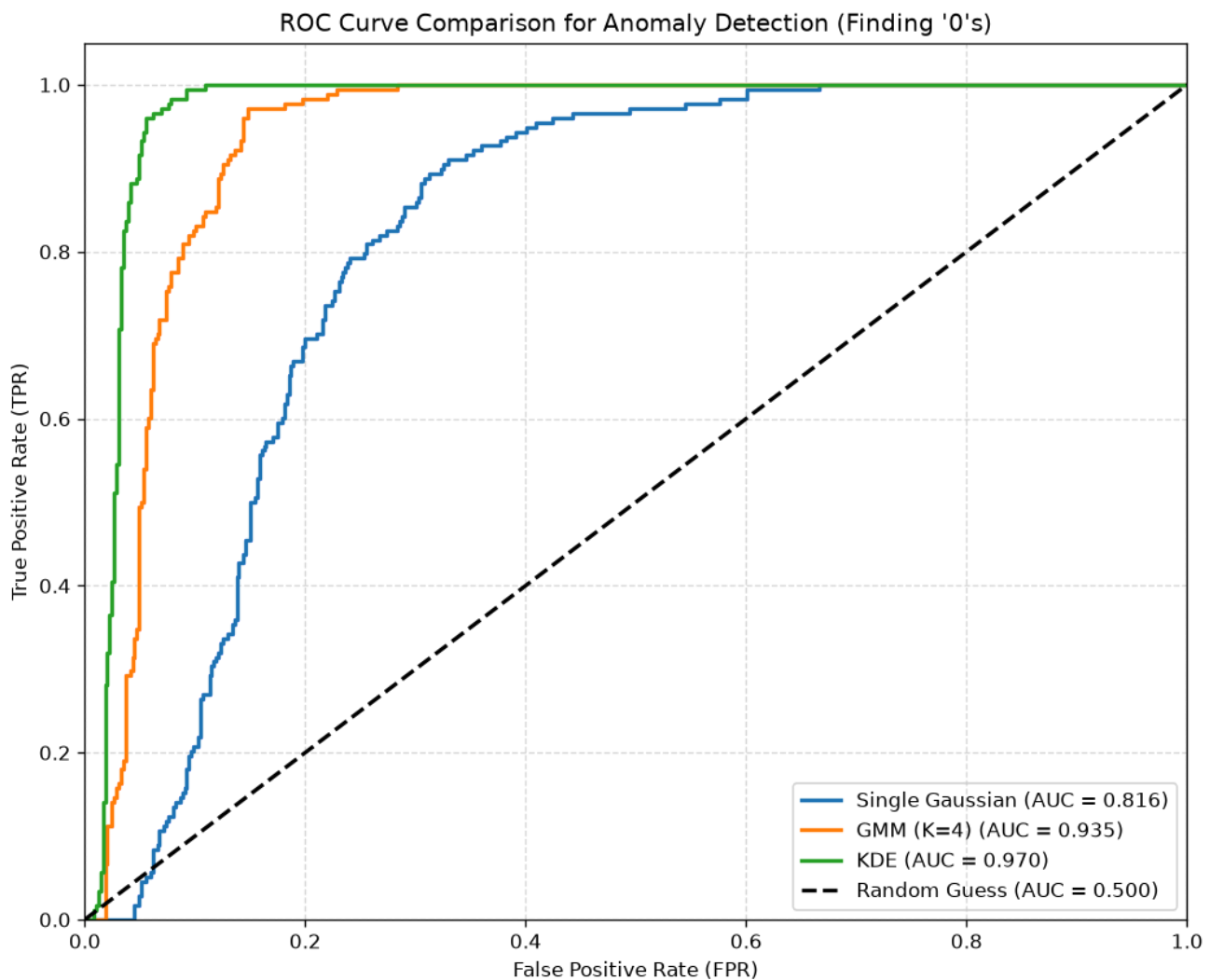
# --- 1. Single Gaussian ---
fpr, tpr, _ = roc_curve(results_df['y_true'], results_df['SingleGaussian_NLL'])
auc_single = auc(fpr, tpr)
plt.plot(fpr, tpr, lw=2, label=f'Single Gaussian (AUC = {auc_single:.3f})')

# --- 2. GMM (K=9) ---
fpr, tpr, _ = roc_curve(results_df['y_true'], results_df['GMM_K4_NLL'])
auc_gmm = auc(fpr, tpr)
plt.plot(fpr, tpr, lw=2, label=f'GMM (K=4) (AUC = {auc_gmm:.3f})')

# --- 3. KDE ---
fpr, tpr, _ = roc_curve(results_df['y_true'], results_df['KDE_NLL'])
auc_kde = auc(fpr, tpr)
plt.plot(fpr, tpr, lw=2, label=f'KDE (AUC = {auc_kde:.3f})')

# --- 4. Plot Styling ---
plt.plot([0, 1], [0, 1], 'k--', lw=2, label='Random Guess (AUC = 0.500)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC Curve Comparison for Anomaly Detection (Finding \'0\'s)')
plt.legend(loc='lower right')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```





We note that the most flexible density estimation model is the KDE in this case. A single Gaussian oversimplifies the problem, while a GMM adds back some flexibility.

Exercises

1. **Bins lie.** Plot a histogram of the same variable with 5, 20 and 100 bins, and with the bin edges shifted by half a bin. How many different stories can one variable tell?
2. **Bandwidth.** Plot kernel density estimates of the same sample with a very small, a very large and a cross-validated bandwidth. Relate the three pictures to the bias-variance tradeoff.
3. **Kernel choice.** Compare Gaussian, Epanechnikov and tophat kernels at the same bandwidth. Which choice matters more, the kernel or the bandwidth?
4. **Parametric versus not.** Fit a single Gaussian and a KDE to a clearly bimodal sample. Plot both. What exactly does the parametric model get wrong, and what does it get right?
5. **Mixtures.** Fit Gaussian mixtures with 1 to 6 components and select the number by BIC. Does the selected model recover the structure you built into the data?
6. **Anomaly detection.** Use a fitted density to flag the 1% least likely points of a dataset. Inspect them. Are they errors, rare cases, or perfectly ordinary points that happen to sit in a sparse region?

References

- https://en.wikipedia.org/wiki/Multivariate_kernel_density_estimation
- https://en.wikipedia.org/wiki/Kernel_density_estimation
- http://faculty.washington.edu/yenchic/18W_425/Lec7_knn_basis.pdf
- Section 9.2 of [1]

[1] Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006.